

提高级 CSP-S 第 6 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每小题仅有一个正确选项)

- 下列 C++ 表达式,值最大的是()。

A. 'Z'-'A' B. $52\%53>>1$
 C. $(\text{rand}()-\text{rand}()+1)\%26$ D. $20+15\%28/3$
- 下列属于解释执行的程序设计语言是()。

A. C B. C++ C. Pascal D. Python
- 对于 64KB 的存储器,其最大地址码用十六进制表示是()。

A. 10000 B. FFFF C. 1FFFF D. EFFFF
- 为解决 Web 应用中的不兼容问题,保障信息的顺利流通(),制定了一系列标准,涉及 HTML、XML、CSS 等,并建议开发者遵循。

A. 微软 B. 美国计算机协会(ACM)
 C. 联合国教科文组织 D. 万维网联盟(W3C)
- 若一个整数是另一个整数的平方,则称该数是“完全平方数”,如 4、9、25 等。下列表达式无法判断该整数是否为完全平方数的是()。

A. $\text{floor}(\text{sqrt}(x)+0.5)=\text{sqrt}(x)$
 B. $\text{int}(\text{sqrt}(x))=\text{sqrt}(x)$
 C. $x/\text{int}(\text{sqrt}(x))=\text{int}(x/\text{int}(\text{sqrt}(x)))$
 D. $\text{int}(\text{sqrt}(x)) * \text{int}(\text{sqrt}(x))=x$
- 将 8 个名额分给 5 个不同的班级,允许有的班级没有名额,有几种不同的分配方案()。

A. 60 B. 120 C. 495 D. 792
- 同时查找 $2n$ 个数中的最大值和最小值,最少比较次数为()。

A. $3(n-2)/2$ B. $4n-2$ C. $3n-2$ D. $2n-2$
- 具有 n 个顶点, e 条边的图采用邻接表存储结构,进行深度优先遍历和广度优先遍历运算的时间复杂度均为()。

A. $O(n^2)$ B. $O(e^2)$ C. $O(ne)$ D. $O(n+e)$
- 两根粗细相同、材料相同的蜡烛,长度比是 21:16,它们同时开始燃烧,18 分钟后,长蜡烛与短蜡烛的长度比是 15:11,则较长的那根蜡烛还能燃烧()。

A. 150 分钟 B. 225 分钟 C. 128 分钟 D. 9 分钟
- 一次数学考试试题由两部分组成,结果全班有 15 人得满分,第一部分做对的有 31 人,第二部分做错的有 19 人,那么两部分都做错的有()。

A. 3 B. 4 C. 6 D. 12
- 设一组初始记录关键字序列为(50, 40, 95, 20, 15, 70, 60, 45),则以初始增量值 $d=4$ 进行希尔排序,第二趟结束后前 4 条记录关键字为()。

A. 15, 20, 50, 40 B. 15, 40, 60, 20 C. 15, 20, 40, 45 D. 15, 20, 40, 50
- 给出以下邻接矩阵,其表示的图是 DAG(有向无环图)的是()。

A. $\begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$ B. $\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}$ C. $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$ D. $\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$
- 下列关于最短路算法的说法正确的是()。

A. 当图中不存在负权回路但是存在负权边时, Dijkstra 算法可以求出源点到所有点的最短路

- B. 当图中不存在负权边时,调用多次 Dijkstra 算法能求出每对顶点间最短路径
 C. 当图中存在负权回路时,调用一次 Dijkstra 算法也一定能求出源点到所有点的最短路
 D. 当图中存在负权边时,Dijkstra+堆优化算法可以求出源点到所有点的最短路
14. 数列 $\{a_n\}$ 是等差数列,首项 $a_1 > 0$, $a_{2020} + a_{2021} > 0$, $a_{2020} * a_{2021} < 0$,则使前 n 项和 $s_n > 0$ 成立的最大项数 n 是()。
- A. 2020 B. 4040 C. 4041 D. 4042
15. 给定长为 n ($n \leq 1000$) 的字符串,每次可以将连续一段回文序列消去,消去后左右两边会接到一起,求最少消去几次能消完整个序列(单个字符也算回文字符串)。设 $f(i, j)$ 表示消去闭区间 $[i, j]$ 内字符串所需要的最小次数,那么当 $1 \leq i \leq j \leq n$ 时,在不考虑回文串的情况下, $f(i, j)$ 的动态规划方程中包含()。
- A. $\min_{i \leq k < j} \{f(i, k) + f(k+1, j)\}$ B. $\min_{i \leq k < j} \{f(i, k) + f(k+1, j)\} + 1$
 C. $\min_{i \leq k < j} \{f(i, k) * f(k+1, j)\}$ D. $f(i, k) + f(k+1, j), i \leq k < j$

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确填√,错误填×;除特别说明外,判断题 1.5 分,选择题 4 分,共计 40 分)

1.

```

01 #include<iostream>
02 #include<cstdio>
03 using namespace std;
04 int L,ans;
05 char a[2002][2002];
06 int cross(int x,int y) {
07     int length=1;
08     if (x<=1 || x>=L) return 1;
09     for (int i=1;;i++) {
10         if (x-i<=0 || x+i>=L+1) return length;
11         else if (a[x-i][y] != a[x+i][y]) return length;
12         else length+=2;
13     }
14 }
15 int down(int x,int y) {
16     int length=1;
17     if (y<=1 || y>=L) return 1;
18     for (int i=1;;i++) {
19         if (y-i<=0 || y+i>=L+1) return length;
20         else if (a[x][y-i] != a[x][y+i]) return length;
21         else length+=2;
22     }
23 }
24 int MAXN(int a,int b){
25     if(a>=b) return a;
26     else return b;
27 }
28 int main() {
29     cin>>L;
30     for (int i=1;i<=L;i++)
  
```

```

31     for(int j=1;j<=L;j++)
32         cin>>a[i][j];
33     int x,y;
34     cin>>x>>y;
35     ans=MAXN(cross(x,y),down(x,y));
36     cout<<ans;
37     return 0;
38 }

```

●判断题

- (1) 第 35 行若改成 MAXN(down(x,y),cross(x,y)),运行结果不变。()
 (2) 第 34 行输入值包含 0 时,程序可能会产生 Runtime Error。()
 (3) 程序输出的 ans 可能等于 0。()
 (4) 当第 34 行输入值 $x>y$ 时,cross(x,y) 返回值必然大于 down(x,y) 返回值。()

●选择题

- (5) 对于输入的 $L \times L$ 的字符矩阵,ans 值最大是()。
 A. $L-1$ B. L C. $2L$ D. L^2
 (6) 若输入 $L=5, x=y=3, a = \{\{abcba\}, \{bcdcb\}, \{cdcdc\}, \{bcdcb\}, \{abcba\}\}$, 则输出是()。
 A. 2 B. 3 C. 5 D. 10

2.

```

01 #include<cstdio>
02 #include<cmath>
03 const int N=1e9;
04 int isnp[50005],p[50005],pcnt;
05
06 inline void getPrime(const int n=sqrt(N)) {
07     isnp[0]=isnp[1]=1;
08     for (register int i=2;i<=n;++i){
09         if (!isnp[i]) p[pcnt++]=i;
10         for (register int j=0;i*p[j]<=n && j<pcnt;++j){
11             isnp[i*p[j]]=1;
12             if (!(i%p[j])) break;
13         }
14     }
15 }
16
17 int main() {
18     getPrime();int n;
19     while (scanf("%d",&n) && n){
20         int _sqrt=sqrt(n),ans=n;
21         for (register int j=0;p[j]<=_sqrt && j<pcnt;++j){
22             if (n%p[j]==0) {
23                 while (n%p[j]==0) n/=p[j];
24                 ans=1ll*ans*(p[j]-1)/p[j];
25             }
26         }

```



```

27     if (n != 1) ans = 1ll * ans * (n - 1) / n;
28     printf("%d\n", ans);
29 }
30 return 0;
31 }

```

● 判断题

- (1) 若去掉第 12 行, 程序也能得到正确结果。()
- (2) 若去掉第 23 行, 程序也能得到正确结果。()
- (3) 若输入的 $n \leq 10^8$, 则第 10 行 $j < \text{pcnt}$ 条件可以省略。()
- (4) 若输入的 $n \leq 10^8$, 则第 21 行 $j < \text{pcnt}$ 条件可以省略。()

● 选择题

- (5) 当 $n = 504$ 时, 输出 ans 为()。
- A. 288 B. 144 C. 504 D. 503
- (6) getPrime 函数的时间复杂度是()。
- A. $O(n)$ B. $O(\log n)$ C. $O(\sqrt{n})$ D. $O(n\sqrt{n})$

3.

```

01 #include<iostream>
02 using namespace std;
03
04 const int inf=0x3f3f3f3f,N=4e5+5;
05 struct edge {
06     int to,nt;
07 }E[N<<1];
08 int cnt,n,m,rt,MX;
09 int H[N],sz[N],mxs[N];
10 void add_edge(int u,int v) {
11     E[++cnt]=(edge){v,H[u]};
12     H[u]=cnt;
13 }
14 void dfs(int u,int fa) {
15     sz[u]=1;
16     for (int e=H[u];e=e[E[e].nt) {
17         int v=E[e].to;
18         if (v==fa) continue;
19         dfs(v,u);
20         sz[u]+=sz[v];
21         mxs[u]=max(mxs[u],sz[v]);
22     }
23     mxs[u]=max(mxs[u],n-sz[u]);
24     if (mxs[u]<mxs[rt]) rt=u;
25     if (mxs[u]==mxs[rt] && u<rt) rt=u;
26 }
27 int main() {
28     cin>>n;
29     rt=cnt=0;
30     for (int i=1;i<n;i++) {

```

```

31     int u,v;
32     cin>>u>>v;
33     add_edge(u,v);
34     add_edge(v,u);
35 }
36 mxs[0]=inf;
37 dfs(1,0);
38 cout<<rt<<' '<<mxs[rt]<<'\n';
39 return 0;
40 }

```

●判断题

- (1)第 33、34 行只需保留任意一行也不会影响程序的正确性。()
 (2)第 37 行函数调用 $\text{dfs}(x,y)$, 只需保证 $1 \leq x \leq n, y \leq 0$ 即可。()
 (3)第 32 行输入若有重复(重边), 不影响输出结果的正确性。()
 (4)程序运行结束时可能存在正整数 $i(i \leq n)$ 使 $\text{sz}[i]$ 等于 $\text{mxs}[i]$ 。()

●选择题

- (5) $n=6$, 二元组 (u,v) 各个值分别是 $\{(1,3), (6,3), (2,6), (5,6), (3,4)\}$, 则输出是()。
 A. 3 2 B. 3 3 C. 6 3 D. 6 2
 (6)若 $n=1000$, 则程序运行后 $\text{mxs}[]$ 数组中除初始值 inf 外, 最大值是()。
 A. 499 B. 500 C. 999 D. 1000

三、完善程序(单选题, 每题 2.5 分, 共计 30 分)

1. (翻硬币)一摞硬币共有 m 枚, 每一枚都是正面朝上。取下最上面的一枚硬币, 将它翻面后放回原处。然后取下最上面的 2 枚硬币, 将它们整体一起翻面后放回原处(如 101111 前 5 个翻面结果是 000101)。再取 3 枚, 取 4 枚……直至 m 枚。然后再从这摞硬币最上面的一枚开始, 重复刚才的做法。这样一直做下去, 直到这摞硬币中每一枚又是正面朝上为止。例如, m 为 1 时, 翻两次即可; m 为 4 时, 翻 11 次; m 为 9 时, 翻 80 次。

【输入】

仅有的一个数字是这摞硬币的枚数 $m(0 < m < 1000)$ 。

【输出】

为了使这摞硬币中的每一枚都是正面朝上所必须翻的次数。

【输入样例】

30

【输出样例】

899

```

01 #include<iostream>
02 using namespace std;
03 int solve(int m);
04 int main(){
05     int m;
06
07     do{
08         scanf("%d", &m);
09         if(m>0 && m<1000)
10             printf("%d\n", ①);
11     } while (m>0 && m<1000);

```

```

12     return 0;
13 }
14
15 int solve(int m){
16     int i,t,s=-1;//翻转的次数
17     int flag;
18     if(m==1)
19         s=②;
20     else{
21         d=2*m+1;
22         //确定硬币是经过偶数次翻转还是奇数次翻转
23         t=2;
24         //表示一个 COIN 必须翻转偶数次,才能从正面继续翻回到正面
25         i=1;//翻转的轮数,每轮为从 1 翻转到 m
26         flag=0; //退出循环标志,翻转完成标志
27         do{
28             if(t==1){
29                 s=③;
30                 flag=1;
31             }else if(④){
32                 s=i*m-1;
33                 flag=1;
34             }else{
35                 t=⑤;
36             }
37             i=i+1;
38         } while(!flag);
39     }
40     return s;
41 }

```

(1)①处应填()。

- A. solve(m+1) B. solve(2 * m)
C. solve(m) D. solve(m)+1

(2)②处应填()。

- A. 0 B. 1 C. 2 D. -1

(3)③处应填()。

- A. i * m B. i * m+1
C. i * m-1 D. i * (m+1)

(4)④处应填()。

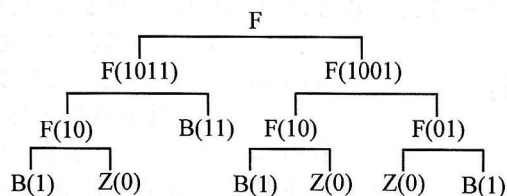
- A. t = 2 * m-1 B. t = 2 * m+1
C. t = 2 * m D. t = 2 * (m+1)

(5)⑤处应填()。

- A. (t * 2+1)%d B. (t * 2)%d
C. (t * 2)%d+1 D. t%d

2. (FBZ 串问题)已知一个由 0,1 字符组成的长度为 2^n 的字符串。请按以下规则分解为 FBZ 串: • 若其中字符全为 1,则称其为 B 串; • 若其中字符全为 0,则称其为 Z 串; • 若不

全为 0,也不全为 1,则称 F 串。若此串为 F 串,则应将此串分解为 2 个长为 2^{n-1} 的子串。对分解后的子串,仍按以上规则继续分解,直到全部为 B 串或为 Z 串为止。例如 $n=3$ 时,给出 0-1 串为“10111001”。



最后输出:FFFBZBFFBZFZB。给定一个 0-1 串,分解成 FBZ 串。

```

01 #include<iostream>
02 #include<cstdio>
03 #include<cstring>
04 using namespace std;
05 const int n=8;
06 char str1[2*n][n];
07 char str2[40]=" ";
08 int main() {
09     int s1,s2,x,s,t;
10     s1=s2=x=0;
11     gets(str1[0]);
12     while ( ① ) {
13         s=t=0;
14         for (int i=0;i<n;i++) {
15             if (str1[s2][i]==' 1')
16                 s++;
17             if (str1[s2][i]==' 0')
18                 t++;
19         }
20         if ( ② )
21             str2[x++]=' B';
22         else if ( ③ )
23             str2[x++]=' Z';
24         else {
25             str2[x++]=' F';
26             int j=(s+t)>>1;
27             for (int k=n*2-2;k>= ④ ;k--)
28                 for (int i=0;i<n;i++)
29                     str1[k+2][i]=str1[k][i];
30             s1+=2;
31             for (int i=0;i<j;i++) {
32                 str1[s2+1][i]=str1[s2][i];
33                 str1[s2+2][i]= ⑤ ;
34             }
35             for (int i= ⑥ ; ⑦ ;i++) {
36                 str1[s2+1][i]=' ';

```



```

37     str1[s2+2][i]=' ';
38 }
39     }
40     s2++;
41 }
42 puts(str2);
43 return 0;
44 }

```

- (1) ①处应填()。
- A. $s1 \leq s2$ B. $s2 \leq s1$ C. $x \leq n$ D. $s \leq t$
- (2) ②处应填()。
- A. $t = 0$ B. $s = 0$ C. $t > 0$ D. $s > 0$
- (3) ③处应填()。
- A. $t = 0$ B. $s = 0$ C. $t > 0$ D. $s > 0$
- (4) ④处应填()。
- A. $s2$ B. $s2+1$ C. $s1$ D. $s1+1$
- (5) ⑤处应填()。
- A. $str1[s2][j]$ B. $str1[s2+1][i]$ C. $str1[s2][i+j]$ D. $str1[s2+1][i+j]$
- (6) ⑥处应填()。
- A. 1 B. j C. s1 D. s2
- (7) ⑦处应填()。
- A. $i < j$ B. $i < s1$ C. $i < s2$ D. $i < n$